

UATU — PRODUCT & DEVELOPMENT SPECIFICATION

v0.1 — Team Mexico Service — Global 2026

1. CONTEXTO

Uatu es un sistema interno de coordinación para **Team Mexico Service** durante FIRST Global.

El objetivo de Team Mexico Service es ayudar a otros equipos durante los matches para maximizar su performance y, por consecuencia, maximizar los puntos en el match.

Ejemplo:

- Un equipo no tiene escalador funcional.
- Team Mexico Service identifica el problema.
- Una persona de mecánica ayuda a reparar o preparar el mecanismo.
- Posteriormente una persona de programación verifica que el mecanismo funcione correctamente.
- La ayuda se marca como completada.
- El sistema registra quién participó, qué se hizo y cuánto tiempo tomó.

Uatu busca reemplazar la organización informal basada principalmente en mensajes de WhatsApp por un sistema estructurado, rápido y visible en tiempo real.

2. PROBLEMA

En años anteriores la coordinación de Team Mexico Service se realizó principalmente mediante mensajes de WhatsApp.

Esto genera varios problemas:

- Los mensajes pueden olvidarse o ignorarse.
- Es difícil saber quién está disponible.

- Es difícil saber quién está ayudando a qué equipo.
- Los mentores no tienen una vista centralizada del equipo.
- Las prioridades pueden cambiar rápidamente.
- Una persona puede terminar una tarea y quedar disponible sin que los demás lo sepan.
- Una persona puede necesitar ayuda adicional.
- Algunas tareas dependen de otras.
- Es difícil saber si una ayuda realmente terminó.
- No existe un historial estructurado de las acciones.
- Es difícil medir tiempos y performance del Service.
- Durante un match la información debe interpretarse rápidamente.

Uatu debe solucionar estos problemas mediante un sistema centralizado de **personas + tareas + estados + equipos + prioridades + matches + historial**.

3. OBJETIVO PRINCIPAL

Uatu debe permitir que Team Mexico Service pueda responder rápidamente a estas preguntas:

1. ¿Quién está disponible?
 2. ¿Quién está ocupado?
 3. ¿Quién está ayudando a qué equipo?
 4. ¿Qué equipo necesita ayuda?
 5. ¿Qué tipo de ayuda necesita?
 6. ¿Cuál es la prioridad de cada equipo?
 7. ¿Qué tareas están pendientes?
 8. ¿Qué tareas están en progreso?
 9. ¿Qué ayudas ya terminaron?
 10. ¿Quién está realizando cada tarea?
 11. ¿Cuánto tiempo lleva cada tarea?
 12. ¿Qué tareas dependen de otras?
 13. ¿Qué matches ya fueron cubiertos?
 14. ¿Qué ocurrió durante el evento?
-

4. CONCEPTO DE UATU

Uatu debe considerarse como un **sistema de coordinación operativa en tiempo real**.

WhatsApp será inicialmente una interfaz de comunicación, pero **WhatsApp NO debe ser el núcleo del sistema.**

Arquitectura conceptual:

UATU CORE

- |— Events
- |— Matches
- |— Teams
- |— Users
- |— Tasks
- |— Priorities
- |— States
- |— Notifications
- |— History
- |— Analytics

Interfaces:

- |— WhatsApp
- |— Dashboard

Toda la lógica importante debe existir en Uatu Core para permitir cambiar o agregar interfaces en el futuro.

5. EVENTO INICIAL

Evento:

- Name: Global 2026
- Location: Incheon, South Korea
- Dates: October 6–10, 2026

6. USUARIOS

Alumnos

Usuario	Role
Raúl	Strategist / Checklist / Programming Service
Abril	Mechanical Service
Isabella	Mechanical Service
Martha	Mechanical Service
Santi	Programming Service
Barbie	Mechanical Service
Danna	Mechanical Service / Pit (Y hacer check list)
Enevi	Mechanical Service / Pit (Y hacer check list)
Ernesto	Mechanical Service

Mentores

Usuario	Role
Mike	Mentor (Progra)
Hugo	Mentor (Mecanica/Progra)
Ivan	Mentor (Mecanica)
David	Mentor (Mecanica)

7. ROLES Y PERMISOS

Inicialmente existirán tres categorías:

Student / Service Member

Puede:

- Ver sus tareas.
- Ver su estado.

- Aceptar tareas.
- Iniciar tareas.
- Completar tareas.
- Solicitar ayuda.
- Cambiar a disponible.
- Consultar información necesaria para realizar su tarea.

Strategist / Coordinator

Puede:

- Crear tareas.
- Asignar tareas.
- Cambiar prioridades.
- Reasignar personas.
- Supervisar matches.
- Ver el estado completo del Service.
- Marcar excepciones.
- Consultar historial.
- Ver el estado completo del equipo.
- Supervisar tareas.
- Modificar/asignar tareas si es necesario.
- Consultar historial.
- Intervenir en situaciones críticas.

Los permisos exactos podrán modificarse posteriormente.

8. ESTADOS DE PERSONA

El sistema debe mantener siempre un estado actual para cada persona.

Estados iniciales:

AVAILABLE

La persona está libre y puede recibir una tarea.

BUSY

La persona está realizando una tarea.

Debe mostrar qué tarea está realizando.

Ejemplo:

Raúl

Status: BUSY

Task: Programming Service — Brazil (1)

REST

La persona está descansando y temporalmente no debe recibir tareas.

NEEDS_HELP

La persona está realizando una tarea pero necesita asistencia.

Posteriormente pueden agregarse otros estados si son necesarios.

9. ESTADOS DE TAREA

Toda tarea debe tener un estado.

Estados iniciales:

PENDING

La tarea existe pero todavía no comenzó.

ASSIGNED

La tarea fue asignada a una persona.

IN_PROGRESS

La persona comenzó la tarea.

BLOCKED

La tarea no puede continuar debido a un problema o dependencia.

COMPLETED

La tarea terminó satisfactoriamente.

10. TRANSICIONES DE TAREA

Flujo básico:

PENDING

→ ASSIGNED

→ IN_PROGRESS

→ COMPLETED

Excepción:

IN_PROGRESS

→ BLOCKED

→ IN_PROGRESS

O:

IN_PROGRESS

→ CANCELLED

Una tarea no debe poder cambiar arbitrariamente entre estados. Uatu debe controlar qué transiciones son válidas.

11. TIPOS DE TAREA

Las tareas iniciales serán:

- PIT
- STRATEGY
- MATCH
- MECHANICAL_SERVICE
- PROGRAMMING_SERVICE

Para Mechanical Service existirán objetivos:

- Mechanical Service — Team 1

- Mechanical Service — Team 2
- ...
- Mechanical Service — Team 7

Para Programming Service:

- Programming Service — Team 1
- Programming Service — Team 2
- ...
- Programming Service — Team 7

El número 1–7 representa la prioridad/posición de servicio, NO necesariamente el número real del equipo.

Esto debe mantenerse separado del nombre del país/equipo.

12. FLUJO PRINCIPAL DE AYUDA

Una ayuda a un equipo normalmente seguirá este flujo:

1. Se identifica un equipo que necesita ayuda.
2. Admin determina su prioridad.
3. Se crea/activa la ayuda.
4. Se asigna una persona.
5. La persona acepta.
6. La tarea pasa a IN_PROGRESS.
7. La persona realiza la intervención.
8. Si el problema es mecánico y requiere programación:
 - Mechanical Service termina.
 - Se genera/activa Programming Service.
9. Programming Service verifica el funcionamiento.
10. Si el sistema funciona correctamente, la ayuda se marca COMPLETED.
11. Las personas involucradas vuelven a AVAILABLE o reciben una nueva tarea.
12. Uatu registra todo el historial.

Ejemplo:

TEAM BRAZIL

Mechanical Service

↓

Mecanismo reparado



Programming Service



Sistema probado



Help Completed

13. DEPENDENCIAS ENTRE TAREAS

Las tareas pueden depender de otras.

Ejemplo:

Mechanical Service



Programming Service



Help Completed

Programming Service no debería activarse antes de que Mechanical Service haya terminado cuando exista una dependencia explícita.

En el futuro podrán existir cadenas más complejas:

Task A



Task B



Task C

Uatu debe almacenar estas dependencias.

14. EQUIPOS

Cada equipo debe tener como mínimo:

- Country

- Alliance
- Match
- Priority
- Required Service
- Current Service Status

Ejemplo:

Brazil

Alliance: Our Alliance

Match: 120

Priority: 1

Required Service: Mechanical & Programming

Status: IN_PROGRESS

15. PRIORIDAD DE EQUIPOS

En cada match existen:

- 2 alianzas
- 3 equipos por alianza
- 6 equipos totales

Team Mexico Service normalmente debe priorizar aproximadamente 5 de esos equipos según necesidad.

La prioridad inicial propuesta es:

PRIORITY 1

Equipo de nuestra alianza del siguiente match — MÁS NECESITADO

PRIORITY 2

Equipo de nuestra alianza del siguiente match — MENOS NECESITADO

PRIORITY 3

Equipo de la otra alianza del siguiente match — MÁS NECESITADO

PRIORITY 4

Equipo de la otra alianza del siguiente match — SEGUNDO MÁS NECESITADO

PRIORITY 5

Equipo de la otra alianza del siguiente match — MENOS NECESITADO

PRIORITY 6

Equipo de nuestra alianza del próximo match — MÁS NECESITADO

PRIORITY 7

Equipo de nuestra alianza del próximo match — MENOS NECESITADO

IMPORTANTE:

La lógica exacta para calcular estas prioridades todavía debe definirse.

Uatu debe separar:

- Team
- Match
- Alliance
- Need
- Priority

No debe guardar simplemente "Team 1", sino almacenar los datos que permiten calcular esa prioridad.

16. PRIORIDAD DINÁMICA

La prioridad puede cambiar durante el evento o recorrerse si uno de los rankeados por prioridad ya se completo se recorrería todas las prioridades.

Ejemplos:

- Un equipo solucionó su problema.
- Un equipo desarrolló un problema nuevo.
- Se acerca su match.
- Un equipo necesita ayuda urgente.
- Una tarea prioritaria quedó bloqueada.

Por lo tanto, la prioridad debe poder modificarse sin destruir el historial anterior.

Idealmente:

Current Priority = 2

History:

14:00 → Priority 4

14:08 → Priority 2

14:15 → Priority 1

17. MATCHES

Cada match debe almacenar:

- Match number
- Scheduled time
- Alliance 1
- Alliance 2
- Teams
- Service status
- Teams helped
- Tasks associated

Ejemplo:

MATCH 120

Teams to help: 5

Status:

COMPLETED

Help:

5/5

18. ESTADO DE UN MATCH

Un match puede mostrar:

NOT_STARTED

0/5 equipos ayudados.

IN_PROGRESS

Ejemplo:

2/5 equipos ayudados.

COMPLETED

5/5 equipos ayudados.

También puede existir:

PARTIAL

El match terminó pero no fue posible ayudar a todos los equipos prioritarios.

Esto permitirá analizar posteriormente por qué no se completó.

19. DASHBOARD

La dashboard será una interfaz para coordinadores y mentores.

La dashboard NO es la prioridad inicial de desarrollo y puede implementarse después del núcleo funcional.

Debe mostrar:

People

Ejemplo:

Raúl — AVAILABLE

Abril — BUSY — Team Brazil

Isabella — BUSY — Team Cambodia

Martha — REST
Santi — BUSY — Programming Service

Active Teams

Mostrar equipos ordenados por prioridad:

Team 1 — Mechanical Service — Abril
Team 2 — Programming Service — Santi + Raúl
Team 3 — COMPLETED
...

Matches

Ejemplo:

Match 120 — COMPLETED — 5/5
Match 160 — IN_PROGRESS — 2/5
Match 200 — NOT_STARTED — 0/5

Timeline

Ejemplo:

14:00 — Ernesto started "Mechanical Service — Brazil"
14:03 — Abril completed "Mechanical Service — Cambodia"
14:07 — Mike requested assistance
14:09 — Raúl assigned task #31

20. MÉTRICAS

Uatu debe guardar suficiente información para calcular posteriormente:

- Average Task Time
- Average Response Time
- Number of Tasks
- Completed Tasks
- Cancelled Tasks
- Blocked Tasks
- Number of Teams Helped
- Teams Helped per Match
- Mechanical Services
- Programming Services
- Average Help Time
- Person Utilization
- Match Coverage
- Unresolved Problems

Ejemplo:

Average Task Time:
2h 14m 32s

21. HISTORIAL / AUDIT LOG

Toda acción importante debe registrarse.

Ejemplo:

14:00

Ernesto → TASK_STARTED

14:03

Abril → TASK_COMPLETED

14:07

Mike → NEEDS_HELP

14:09

Raúl → TASK_ASSIGNED

El historial debe conservar:

- Timestamp
- User
- Action
- Task
- Team
- Match
- Previous State
- New State

Esto permitirá reconstruir lo ocurrido durante el evento.

22. WHATSAPP

La primera interfaz para los alumnos será WhatsApp mediante Meta WhatsApp Business Platform / Templates.

WhatsApp será una interfaz, no el sistema central.

Flujo:

Uatu Core

→ WhatsApp

→ User

Y:

User

→ WhatsApp

→ Uatu webhook

→ Uatu Core

→ Database

→ Dashboard / Notifications

23. MENSAJES PRINCIPALES DE WHATSAPP

Inicialmente se necesitan pocos flujos.

New Task

Uatu:

Nueva tarea asignada.

Team:

Brazil

Service:

Mechanical

Priority:

1

[ACCEPT]

[CAN'T DO]

Accept

Usuario:

[ACCEPT]

Uatu:

✓ Task accepted.

Status:

IN_PROGRESS

Need Help

Usuario:

[NEED HELP]

Uatu:

¿Qué necesitas?

- Mechanical
- Programming
- Another person
- Material
- Other

Complete

Usuario:

[COMPLETE]

Uatu:

✓ Help completed.

[AVAILABLE]

[NEW TASK]

24. PRINCIPIO DE DISEÑO DE WHATSAPP

Las interacciones deben requerir la menor cantidad posible de texto manual.

Durante un evento las personas estarán ocupadas.

Preferir:

Buttons

→ Quick Replies

→ Templates

→ Structured selections

Evitar:

"Escribe exactamente qué quieres hacer..."

El sistema debe permitir actualizar un estado en pocos segundos.

25. UATU CORE

La arquitectura lógica inicial debe separarse en módulos:

- Authentication
- Users
- Roles
- Events
- Matches
- Teams
- Tasks
- Priorities
- Task Dependencies
- Notifications
- WhatsApp
- Activity Log
- Analytics

La lógica de negocio debe estar en el backend y no directamente en WhatsApp o en el frontend.

26. MODELO DE DATOS INICIAL

Entidades principales:

User

- id
- name
- role
- serviceType
- status

Event

- id
- name
- location
- startDate
- endDate
- status

Team

- id
- country
- teamNumber

Match

- id
- matchNumber
- scheduledTime
- status

Task

- id
- type
- status
- priority
- assignedUser
- team
- match
- createdAt
- startedAt
- completedAt

TaskDependency

- taskId
- dependsOnTaskId

TaskHistory

- id
- taskId
- userId
- action
- timestamp

TeamPriority

- teamId
 - matchId
 - priority
 - reason
-

27. REGLAS IMPORTANTES DEL SISTEMA

Estas reglas deben formar parte del backend:

1. Una persona puede tener una o varias tareas solamente si el sistema lo permite explícitamente.
 2. Una persona BUSY no debe recibir otra tarea normal.
 3. Una persona AVAILABLE puede recibir tareas.
 4. Una persona REST no debe recibir tareas.
 5. Una tarea COMPLETED no puede volver a IN_PROGRESS sin una acción especial.
 6. Una tarea BLOCKED debe registrar el motivo.
 7. Una tarea dependiente no puede comenzar antes de que su dependencia se complete.
 8. Cambiar una prioridad debe generar historial.
 9. Completar una tarea debe registrar timestamp.
 10. Cambiar el estado de una persona debe registrarse.
 11. Una ayuda debe estar relacionada con un Team.
 12. Una ayuda debe poder relacionarse con un Match.
 13. Las personas involucradas en una ayuda deben quedar registradas.
 14. Los datos históricos no deben eliminarse simplemente porque cambie el estado actual.
-

28. MVP

El primer objetivo NO es construir todo Uatu.

El MVP debe demostrar que el sistema puede coordinar correctamente una ayuda real.

MVP — Fase 1

Users:

- Crear usuarios.

- Roles.
- Estados.

Teams:

- Crear equipos.
- Asignarlos a matches.

Tasks:

- Crear tarea.
- Asignar persona.
- Cambiar estado.
- Completar tarea.
- Registrar historial.

Core workflow:

Task

- Assign
- Accept
- In Progress
- Complete

Y:

Mechanical

- Programming
- Completed

WhatsApp:

- Task assigned
- Accept
- Need help
- Complete
- Available

Dashboard mínima:

- Personas
 - Tareas
 - Equipos
 - Estado
-

29. ROADMAP DE DESARROLLO

El desarrollo debe hacerse incrementalmente.

PHASE 0 — Product Definition

Objetivo:

Definir completamente el comportamiento antes de programar.

Entregables:

- Product specification
 - User stories
 - Roles
 - States
 - Task types
 - Priority rules
 - Match logic
 - Task dependencies
 - WhatsApp flows
 - Error cases
 - MVP scope
-

PHASE 1 — Uatu Core

Objetivo:

Construir el núcleo sin depender de WhatsApp.

Desarrollar:

- Database
- Users
- Roles
- Events
- Teams
- Matches
- Tasks
- States
- Task dependencies
- Task history

- Basic API

Al terminar:

Debe ser posible simular todo un evento mediante API.

PHASE 2 — Task Engine

Objetivo:

Construir correctamente la lógica de coordinación.

Desarrollar:

- State machine
- Task assignment
- Person availability
- Task dependencies
- Priority changes
- Blocking
- Completion
- Automatic status changes
- Event logging

Al terminar:

Uatu debe saber quién está haciendo qué en cualquier momento.

PHASE 3 — WhatsApp Integration

Objetivo:

Permitir utilizar Uatu desde WhatsApp.

Desarrollar:

- Meta integration
- Webhooks
- Templates
- User identification
- Task notifications
- Accept

- Complete
- Need Help
- Available
- Error handling

Al terminar:

Un usuario debe poder completar el ciclo completo sin entrar a la dashboard.

PHASE 4 — Basic Dashboard

Objetivo:

Dar a coordinadores y mentores visibilidad del sistema.

Mostrar:

- People
 - Current status
 - Active tasks
 - Teams
 - Priorities
 - Matches
 - Alerts
 - Activity timeline
-

PHASE 5 — Event Optimization

Agregar:

- Automatic reminders
 - Overdue tasks
 - Priority alerts
 - Blocked-task alerts
 - Match countdown
 - Automatic task suggestions
 - Better dependency handling
-

PHASE 6 — Analytics

Agregar:

- Average task time
 - Average response time
 - Teams helped
 - Help success rate
 - Match coverage
 - Mechanical vs Programming services
 - Person workload
 - Bottlenecks
 - Historical reports
-

PHASE 7 — Intelligence

Una vez que Uatu tenga suficiente información histórica:

- Recommend task assignments.
- Detect bottlenecks.
- Recommend priority changes.
- Detect unusual delays.
- Predict which teams may require assistance.
- Recommend which Service member should handle a task.
- Summarize the event automatically.

La inteligencia artificial debe ser una capa sobre el sistema estructurado, NO el sistema principal de coordinación.

30. ORDEN RECOMENDADO DE DESARROLLO

NO comenzar por:

- UI bonita.
- Analytics.
- AI.

- Automatizaciones complejas.

Comenzar por:

1. Data model
2. States
3. State transitions
4. Task engine
5. Priority logic
6. Match logic
7. API
8. WhatsApp
9. Dashboard
10. Analytics
11. AI

La prioridad es que **el núcleo sea correcto antes de hacerlo bonito.**

31. CRITERIOS DE ÉXITO DEL MVP

El MVP debe poder cumplir este escenario:

1. Raúl crea una ayuda para Brazil.
2. Brazil tiene prioridad 1.
3. Abril recibe Mechanical Service.
4. Abril acepta.
5. Abril pasa a BUSY.
6. La dashboard muestra a Abril trabajando con Brazil.
7. Abril termina la parte mecánica.
8. Uatu detecta que Programming Service es necesario.
9. Santi recibe Programming Service.
10. Santi acepta.
11. Santi pasa a BUSY.
12. Santi termina.
13. Brazil pasa a COMPLETED.
14. Abril y Santi vuelven a AVAILABLE.
15. El match actualiza su progreso.
16. El historial registra todas las acciones.
17. Los mentores pueden ver todo desde la dashboard.

Si este escenario funciona correctamente, tenemos una primera versión funcional de Uatu.

32. PRINCIPIOS DE DESARROLLO

1. Simplicidad durante el evento

Cada acción debe requerir pocos segundos.

2. Estado como fuente de verdad

El sistema debe saber siempre:

Person → Current State → Current Task

3. Historial inmutable

No borrar eventos importantes.

4. WhatsApp como interfaz

No poner la lógica principal dentro de WhatsApp.

5. Dashboard como centro de supervisión

La dashboard debe mostrar el estado actual, no convertirse en otro sistema paralelo.

6. Mobile-first para usuarios

Los alumnos estarán utilizando teléfonos durante el evento.

7. Diseño para fallos

Uatu debe contemplar:

- Internet lento.
- Mensaje no entregado.
- Usuario que no responde.
- Usuario que rechaza tarea.
- Tarea bloqueada.
- Cambio de prioridad.
- Match adelantado/retrasado.
- Persona que queda indisponible.

8. Evitar overengineering

El primer evento sólo tiene aproximadamente 13 usuarios.

La arquitectura debe ser escalable, pero el MVP debe mantenerse simple.

33. PRÓXIMO PASO

Antes de comenzar a programar, Uatu necesita convertirse en una especificación técnica más detallada.

El siguiente trabajo recomendado es construir, en este orden:

1. **User Stories**
2. **State Machine completa**
3. **Flujos de ayuda**
4. **Reglas exactas de prioridad 1–7**
5. **Modelo de base de datos**
6. **API endpoints**
7. **WhatsApp conversation flows**
8. **Dashboard wireframe**
9. **Arquitectura del proyecto**
10. **Stack tecnológico**
11. **MVP checklist**
12. **Roadmap de implementación por tareas**
13. **Testing plan**
14. **Deployment plan**

El objetivo final de esta documentación es que un LLM pueda recibir el documento y entender rápidamente **qué es Uatu, cómo funciona, cuáles son sus reglas y qué debe modificar sin tener que leer información irrelevante.**

Toda implementación futura debe tratar este documento como la fuente principal de verdad del comportamiento de Uatu.